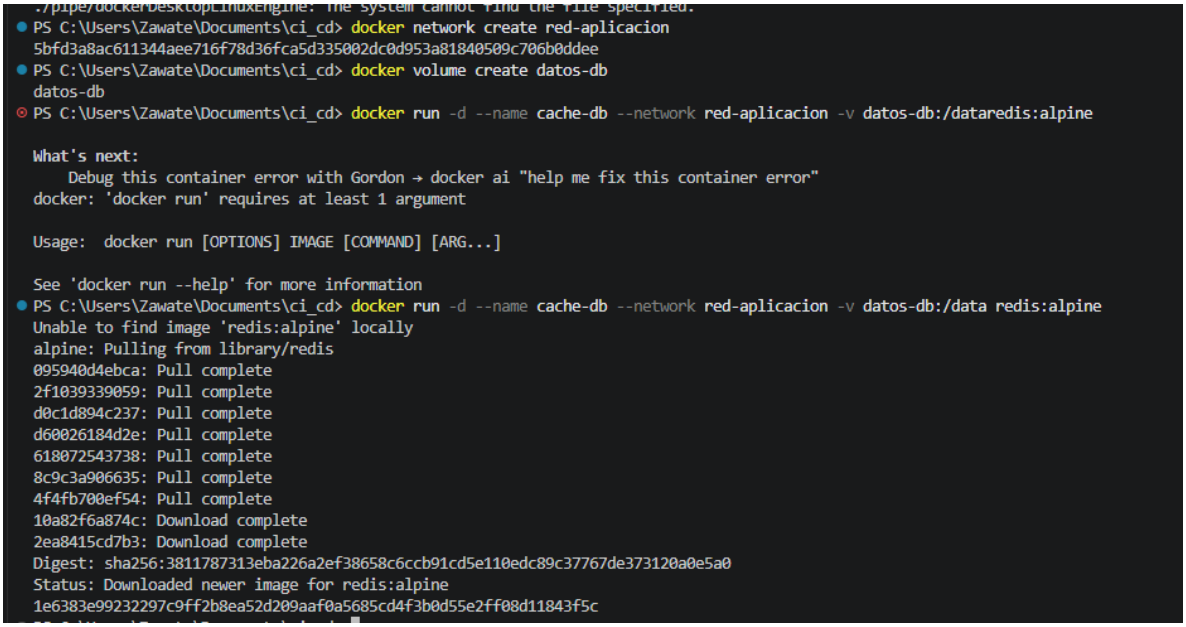
Fase 1: Redes Aisladas y Volúmenes de Almacenamiento



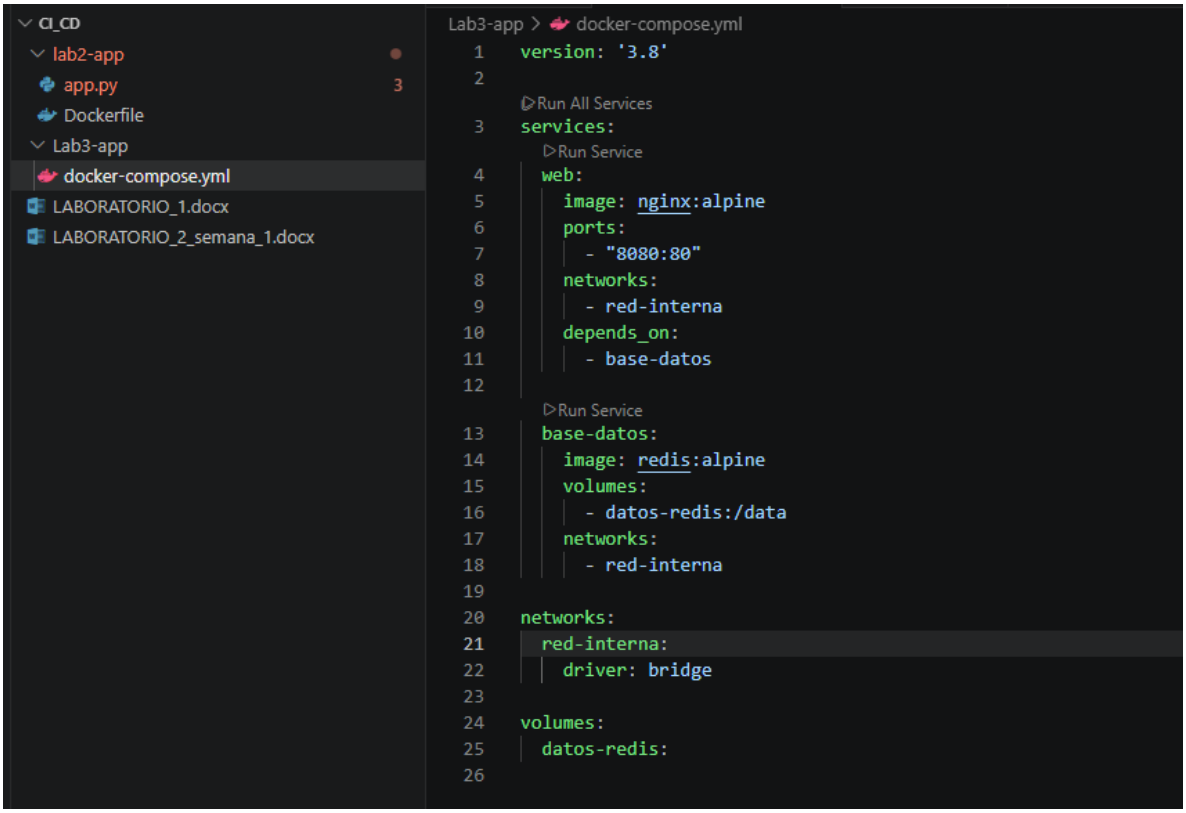
```
.//pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\Zawate\Documents\ci_cd> docker network create red-aplicacion
5bfd3a8ac611344aee716f78d36fca5d335002dc0d953a81840509c706b0ddee
PS C:\Users\Zawate\Documents\ci_cd> docker volume create datos-db
datos-db
PS C:\Users\Zawate\Documents\ci_cd> docker run -d --name cache-db --network red-aplicacion -v datos-db:/dataredis:alpine

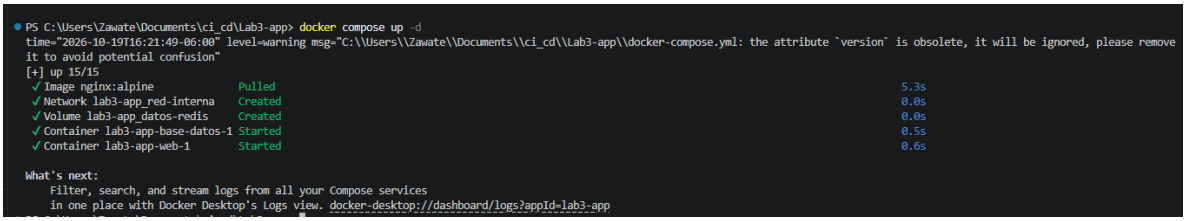
What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
docker: 'docker run' requires at least 1 argument

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

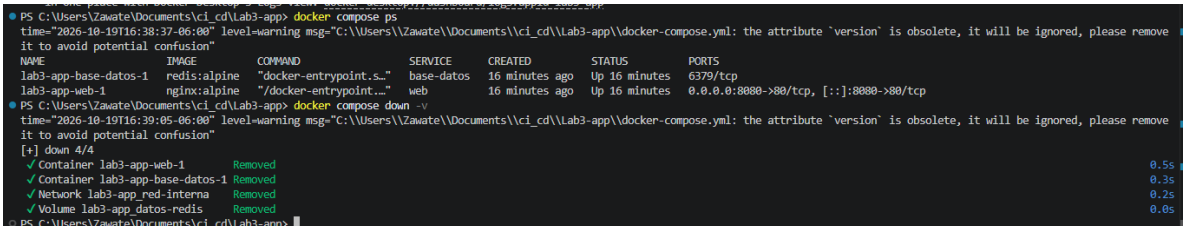
See 'docker run --help' for more information
PS C:\Users\Zawate\Documents\ci_cd> docker run -d --name cache-db --network red-aplicacion -v datos-db:/data redis:alpine
Unable to find image 'redis:alpine' locally
alpine: Pulling from library/redis
095940d4ebca: Pull complete
2f1039339059: Pull complete
d0c1d894c237: Pull complete
d60026184d2e: Pull complete
618072543738: Pull complete
8c9c3a906635: Pull complete
4f4fb700ef54: Pull complete
10a82f6a874c: Download complete
2ea8415cd7b3: Download complete
Digest: sha256:3811787313eba226a2ef38658c6ccb91cd5e110edc89c37767de373120a0e5a0
Status: Downloaded newer image for redis:alpine
1e6383e99232297c9ff2b8ea52d209aaf0a5685cd4f3b0d55e2ff08d11843f5c
```

Fase 2: Orquestación Multi-servicio con Docker Compose





Fase 3: Monitoreo, Escalado y Destrucción Controlada



¿Qué ventajas ofrece el uso de Docker Compose frente a la ejecución individual de múltiples comandos docker run?

https://www.youtube.com/watch?v=evMZ6ipRO2A&t=99s

El uso de **Docker Compose** ofrece tres ventajas principales sobre la ejecución manual de múltiples comandos individuales:

- **Configuración centralizada y declarativa:** En lugar de recordar y escribir comandos larguísimos con decenas de parámetros en la terminal (como puertos, volúmenes y variables de entorno), todo tu entorno se define en **un único archivo YAML** legible y fácil de versionar con Git.
- **Orquestación en un solo comando:** Con un único comando (docker compose up), Docker se encarga de crear las redes, los volúmenes y levantar todos los contenedores en el orden correcto. Para apagarlos y borrar todo el entorno limpio, solo usas docker compose down.
- **Gestión de dependencias:** Permite usar la directiva depends_on (como la que tienes en tu archivo). Esto asegura que los servicios se inicien en el orden adecuado (por ejemplo, que la base de datos de Redis arranque antes que el servidor web Nginx).

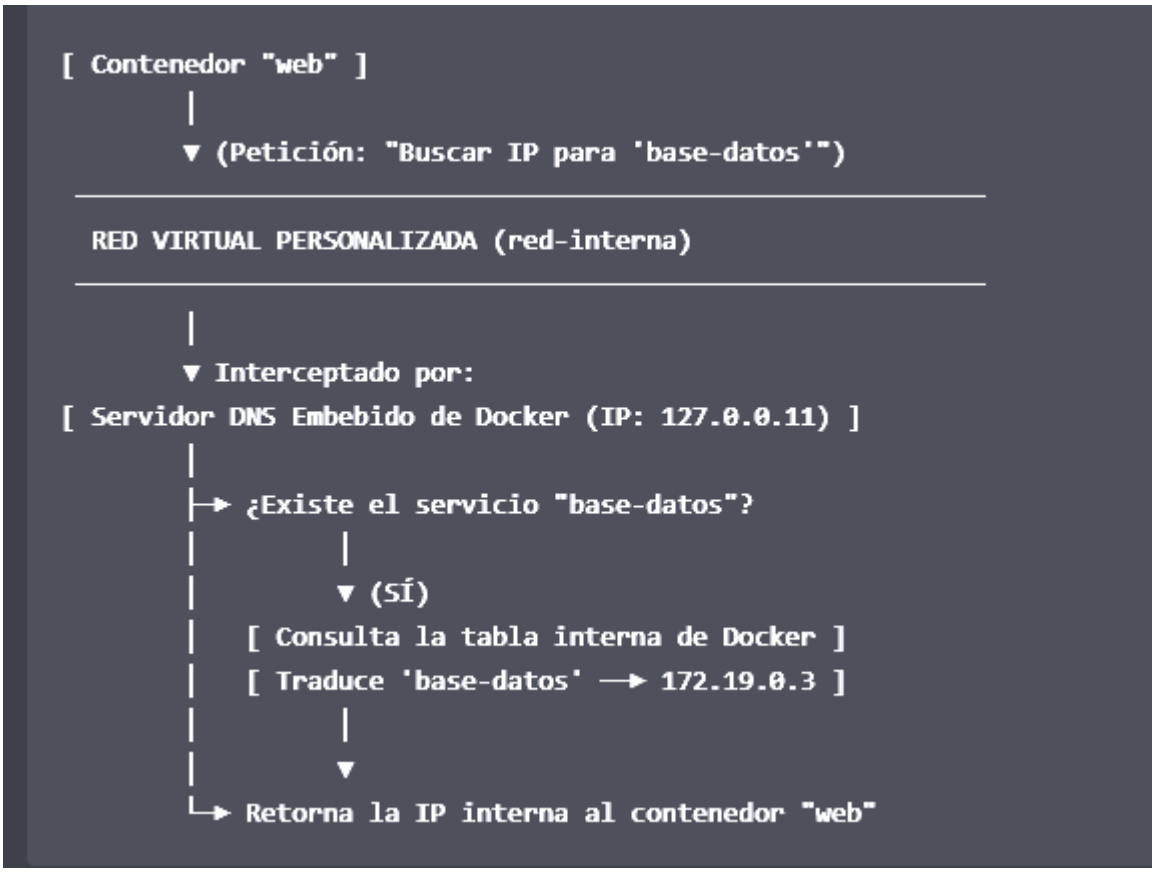
Explique cómo funciona la resolución de nombres DNS interna entre contenedores conectados a la misma red virtual.

https://docs.docker.com/engine/network/

https://www.docker.com/blog/how-docker-desktop-networking-works-under-the-hood/

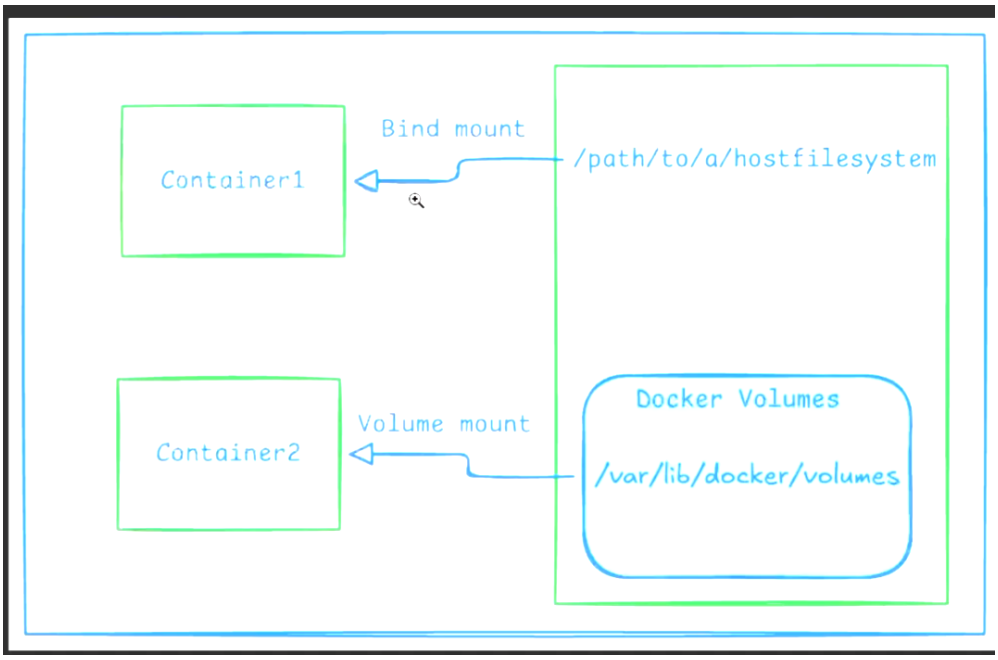
Cuando conectas varios contenedores a la misma **red virtual personalizada** (como tu red red-interna), Docker activa un **servidor DNS interno integrado**:

- **Asignación automática:** Docker registra automáticamente el **nombre del servicio** (o el nombre del contenedor) como un nombre de dominio local en ese servidor DNS.
- **Resolución entre contenedores:** Si tu contenedor web necesita comunicarse con base-datos, no necesita conocer su dirección IP dinámica. El código dentro de web simplemente hace una petición a http://base-datos.
- **Traducción de IP:** El servidor DNS de Docker intercepta esa petición, traduce el nombre base-datos a la IP privada actual del contenedor de Redis y permite la comunicación directa y segura dentro de la red virtual.



¿Cuál es la diferencia entre un volumen administrado (docker volume) y un montaje de directorio local (bind mount)?

https://www.youtube.com/watch?v=cc8FnGMiQNA&t=5s



Característica	Volumen Administrado (docker volume)	Montaje de Directorio (bind mount)
Definición	Docker crea y gestiona una carpeta dedicada dentro del sistema de archivos interno de Docker (fuera del control directo del usuario).	Mapea una **carpeta o archivo específico y existente** de tu computadora local directamente dentro del contenedor.
Control	**Gestionado por Docker**. El usuario no suele modificar los archivos directamente desde el sistema operativo anfitrión.	**Gestionado por el usuario**. Tienes acceso total y directo a los archivos desde tu explorador o editor de código.
Caso de uso principal	Ideal para **producción y persistencia de datos** (bases de datos como Redis o MySQL), ya que son más seguros, tienen mejor rendimiento en la nube y son fáciles de respaldar.	Ideal para **desarrollo local**. Permite modificar el código fuente en tu editor y ver los cambios reflejados en el contenedor en tiempo real sin reiniciar.